

Monash University

FIT5202 - Data processing for Big Data (S1 2026)

Assignment 1: Analysing Road Safety Data

Due Date: 23:55 Friday 17/April/2026 (End of week 6)

Weight: 15% of the final marks

Background: Navigating Road Safety Through Big Data

Road safety remains one of the most pressing public health and infrastructure challenges globally. With the advancement of digital record-keeping, telematics, and emergency response networks, every traffic incident now generates a massive digital footprint. We are no longer just analysing simple incident logs; modern road safety data encompasses a vast array of information, including precise geospatial coordinates, granular weather and environmental conditions, complex vehicle specifications, and detailed demographic profiles.

The sheer volume, variety, and velocity of this information have pushed road safety analysis firmly into the realm of Big Data. Traditional relational databases and standard analytical methods often struggle to scale when processing millions of multidimensional records. To understand the true underlying causes of traffic incidents, researchers must look beyond isolated events and analyse complex, overlapping factors—such as how sudden weather changes correlate with specific vehicle types and driver demographics across diverse geographic zones.

Big Data processing frameworks provide the distributed computing power necessary to handle this complexity. By leveraging these modern tools, data engineers and analysts can:

- **Process at Scale:** Ingest, clean, and standardise massive, messy datasets across distributed clusters.
- **Execute Complex Data Fusions:** Perform multi-attribute joins to merge disparate data sources (e.g., linking individual casualty severities to specific vehicle types and local geographical data) without crashing the system.
- **Uncover Hidden Patterns:** Execute parallel aggregations to identify high-risk "hotspots," temporal accident trends, and vulnerable road user profiles.

In this assignment, you will utilise Apache Spark to analyse a large-scale historical road safety dataset. You will be tasked with parsing raw files, handling missing or invalid data, and writing optimised queries to perform multivariate analysis. By applying appropriate data partitioning and parallel processing strategies, your goal is to transform raw, fragmented incident data into actionable insights that could inform public policy, improve road infrastructure, and ultimately save lives.

The Dataset

The dataset can be downloaded from Moodle, it was derived from a mixture of datasets from UK's road accidents.

You will find the following files after extracting the zip file:

- 1) accidents.csv: Contains casualty data in accidents(collisions)
- 2) collision.csv: Collisions case information
- 3) vehicle.csv: Vehicle information about the collision
- 4) metadata.xlsx: Metadata about different columns in excel format.
- 5) police_force.json: id and name mapping of the police names.

More information and a brief description of the columns are in the appendix.

Assignment Information

In this assignment, you are required to implement various solutions based on DataFrames and Spark SQL in PySpark for the given queries related to the use case.

Getting Started

- Download your dataset from Moodle.
- Download a template file for submission purposes:
 - **A1_template.ipynb** file in Jupyter notebook to write your solution. Rename it into the format (for example: **A1_xxx0000.ipynb**. This file contains your code solution (xxx0000 is your auth code found in your student email account).
- For this assignment, you will use Python 3+ and PySpark 4.x.x. (The environment is provided as a Docker image, the same one you use in labs.)

Part 1: Data Loading and Transformation (15%)

In this section, you need to create data frames from the given datasets, perform partitioning and use various operations to answer the queries.

1.1 Data Preparation and Loading (5%)

1. Write the code to create a SparkContext object using SparkSession. Use the UK (Europe/London) time as the session time zone. Give your application an appropriate name and run Spark locally with **4 cores on your machine**.
2. Load the CSV and JSON files into multiple data frames with an explicit StructType schema (most of the columns can use an integer, except for those with text data and latitude/longitude).
3. Display the total row count and the first 5 records of the collision, accident and vehicle.

1.2 Data Cleansing (10%)

Real-world data is messy. You must filter out anomalies and invalid entries before analysis, please perform the following data cleansing tasks.

1. **Handle Missing/Invalid Values(2%):** Filter the DataFrames to remove any records where key IDs are null and where `did_police_officer_attend_scene_of_accident` contains the missing data code - 1.
2. **Spatial Bounding Box(2%):** The UK has specific geographical bounds. Filter the collision DataFrame to remove any geospatial outliers (accidents that mistakenly plot outside the UK or have missing coordinates). Keep only records where:
 - latitude is between 49.0 and 61.0
 - longitude is between -9.0 and 2.0Instead of hardcoding those values, create a reusable UDF function called **isUK** to perform the filtering.
3. Perform the following subtasks:
 - a) Check and **print** the number of partitions for the data frames. (2%)
 - b) Explain how PySpark determines the default number of partitions when reading a CSV file from a local file system. (2%)
 - c) If we plan to run heavy aggregations grouping by year, month and weather conditions, what strategies should we use to partition our data? (2%)

Part 2. Working with DataFrames (50%)

In this section, you need to use *DataFrame functions* to answer the queries.

Implement the following queries using **dataframes**. You need to be able to perform operations like transforming, filtering, sorting, joining and groupBy using the functions provided by the DataFrame API. **For each task, display the first 5 results where no output is specified.** (Tasks 1-8 are 5% each, task 9 is 10%).

1. The date column is currently a String (DD/MM/YYYY). Using native PySpark functions, convert this column into a proper PySpark DateType. Then, extract the Year and Month into two new integer columns: year and month. Show the first 5 rows of these specific columns.
2. Perform a join between the **collision** data frame and the **police_force** ID mapping, and add the string name of the police force to your collision data frame.
3. Filter the collision data frame to isolate incidents that occurred during "darkness" (using `light_conditions`) **AND** while it was "raining or snowing" (`weather_conditions`). Calculate the ratio of severe/fatal collisions to slight collisions under these exact conditions and compare it to the same ratio during clear daylight.
4. Use the vehicle data frame to find young drivers (aged 17-21) driving small-

engine cars (engine_capacity_cc <= 1600). What are the top 3 most common vehicle_manoeuvre types for this demographic that resulted in the vehicle leaving the carriageway (vehicle_leaving_carriageway not equal to 0)?

5. Join the accidents (casualties) table with the vehicle table to identify which specific vehicle_type (by string name) is statistically most likely to be involved when a pedestrian (casualty_class = 3) is fatally injured.
6. Join tables and perform aggregations: add two columns to the collision dataframe: number_of_vehicles, number_of_casualties. Identify multi-vehicle collisions (3 or more vehicles) and compare the average age of the driver in the *initiating* vehicle (vehicle_reference = 1) versus the driver in the *last* vehicle to be struck (vehicle_reference = maximum for that specific collision_index).
7. Group the data to find the total number of collisions for each police_force and road_type. Print the highest road_type for each police_force.
8. Group collisions by local_authority_district, and create a custom "Danger Index" for each district calculated as: (Count of Fatal * 5) + (Count of Serious * 3) + (Count of Slight * 1) / Total Collisions, print the top 10 most dangerous districts.
9. (Open Question) Explore the dataset freely and plot **two diagrams of your choice**. Which attributes are highly correlated with the number and severity of accident casualties? Discuss the steps of your exploration and the results. (No word limit, please keep concise.)

Part 3: Query Optimisation (35%)

Query (Severe accident involving young people and relatively new cars):

For each year, find the top 5 accidents by the number of involved people with the majority (i.e. > 50%) of casualties in their early 20s (age between 20-25), caused by (i.e. initiating vehicle) relatively new cars (< 3 years), and show the number and percentage of serious and fatal casualties. (Note: if the age of the car is unknown, treat the record as a new car).

3.1 Implement the above query using the DataFrame API and Spark SQL **separately**. Log the time taken for each query in each approach using the “%%time” built-in magic command in Jupyter Notebook. **Discuss the performance difference between these 2 approaches (200 words)**. (Note: You can write a multi-step query or a single complex query; the choice is yours; however, the measured time should be the same.) (10 marks)

3.2 Explore the DAG of the DataFrame API in Spark UI, analyse and identify the potential bottleneck of your query. Include **1-2 annotated** screenshots to highlight the identified bottlenecks and your suggestions for improving query processing speed

(300 words). (10 marks).

3.3 Implement your suggested improvements in 3.2, run the query, measure the time using the built-in magic command and showcase the improvements. (15 Marks)

- You shall include 1-2 annotated screenshots corresponding to the bottlenecks and discuss how your implementation improved them (no word limit, but please keep it concise)
- This task is merit-based. To get full marks, you need to achieve at least a 30% performance improvement while running on the same Spark configuration; for example, reducing the query runtime from 30 seconds to 20 seconds.

Submission

Submit your final assignment solution via Moodle. You must submit the following files. Both files are required for marking; failure to submit either file may incur a penalty of up to 10%.

- Your Jupyter notebook file (e.g., **A1_authcate.ipynb**).
- **A PDF file** saved from jupyter notebook with all output following the file naming format as follows: **A1_authcate.pdf**

Note that both submitted (Jupyter and PDF) files will be scanned using plagiarism detection software. The highest similarity score among students may be interviewed to prove the originality of the work.

Assignment Marking Rubric

For each task individually, you'll be marked based on the quality of your work on a 3-level scale (0%, 50% and 100%).

- 0%: No attempt or incorrect answer with poor attempt;
- 50%: Partial mark for a good attempt but incorrect result;
- 100%: Full mark for correct attempt.

In your submission, the Jupyter notebook file should contain the **code and its output**. It should follow *programming standards, readability of the code, and organisation of code*. Please find the PEP 8 -- Style Guide for Python Code for your reference. Here is the link: <https://peps.python.org/pep-0008/> **A penalty of up to 10% applies if your code is hard to understand due to insufficient comments.**

Late submissions

Late Assignments or extensions will not be accepted unless you submit a special consideration form. ALL Special Consideration, including within the semester, is now to be submitted centrally. This means that students **MUST** submit an online Special Consideration form via Monash Connect. For more details, please refer to the **Unit Information** section in Moodle.

A late submission is subject to a 5% penalty per day, including weekends. The cut-off date is

7 days after the due date. No submission (i.e. 0 mark) will be accepted after the cut-off date unless you have a special consideration.

If you submit the wrong file and ask for the submission to be reopened, the new resubmission timestamp will be used to calculate the late penalty.

Mark Release and Review

- Mark will be released within 10 business days after the submission deadline.
- Reviews and disputes regarding the mark will be accepted a maximum of 7 days after the release date (including weekends).

Other Information

Where to get help

You can ask questions about the assignment in the Assignments section in the Ed Forum accessible on the unit's Moodle Forum page. This is the preferred venue for assignment clarification-type questions. **Students are expected to check the Ed Forum regularly, as responses from the teaching staff are considered official and may constitute amendments or additions to the assignment specification. Clarifications, updates, and additional information will be posted there to help avoid confusion when completing the assignment. Failure to check the forum will not be accepted as a reason for misunderstanding the assignment requirements.**

Also, you can attend scheduled consultation sessions if the problem and the confusion are still not solved.

Plagiarism and collusion

Plagiarism and collusion are serious academic offences at Monash University. Students must not share their work with any other students. Students should consult the policy linked below for more information.

<https://www.monash.edu/students/academic/policies/academic-integrity>

See also the video linked on the Moodle page under the Assignment block.

Students involved in collusion or plagiarism will be subject to disciplinary penalties, which can include:

- The work is not being assessed
- A zero grade for the unit
- Suspension from the University
- Exclusion from the University

Generative AI Statement

As per the University's [policy](#) on the guidelines and practices pertaining to the usage of Generative AI:

AI & Generative AI tools may be used SELECTIVELY within this assessment.

Where used, AI must be used responsibly, clearly documented and appropriately acknowledged (see Learn HQ).

Any work submitted for a mark must:

- 1) Represent a sincere demonstration of your human efforts, skills and subject knowledge that you will be accountable for.
- 2) Adhere to the guidelines for AI use set for the assessment task.
- 3) Reflect the University's commitment to academic integrity and ethical behaviour.

Inappropriate AI use and/or AI use without acknowledgement will be considered a breach of academic integrity.

The teaching team encourages students to apply their own critical thinking and reasoning skills when working on assessments, with assistance from GenAI. Generative AI tools may produce inaccurate content, which could negatively impact students' comprehension of big data topics.

Data source acknowledgement:

The dataset is a remix based on several real-world datasets.

Metadata:

Collision:

Column Name	Description
collision_index	A unique identifier for the collision incident.
longitude	The longitudinal coordinate of the collision location.
latitude	The latitudinal coordinate of the collision location.
police_force	An ID representing the police force that responded to or reported the collision.
collision_severity	The overall severity of the collision (e.g., 1 = Fatal, 2 = Serious, 3 = Slight).
date	The date the collision occurred (DD/MM/YYYY).
time	The time the collision occurred (HH:MM).
local_authority_district	An ID for the local authority district where the collision took place.
road_type	The type of road (e.g., Roundabout, One-way street, Dual carriageway).
speed_limit	The legal speed limit at the location of the collision (in mph).
junction_detail	Physical details of the junction if the collision occurred at or near one.
junction_control	The method of traffic control at the junction (e.g., Give way, Traffic signals).
pedestrian_crossing	Indicates the presence and type of a pedestrian crossing within 50 meters.
light_conditions	The lighting conditions at the time of the collision (e.g., Daylight, Darkness).
weather_conditions	The weather conditions at the time of the collision (e.g., Fine, Raining, Snowing).
road_surface_conditions	The state of the road surface (e.g., Dry, Wet/damp, Frost/ice).

special_conditions_at_site	Any unusual conditions at the site (e.g., Roadworks, Oil/diesel, Mud).
carriageway_hazards	Identifies any objects or hazards on the carriageway (e.g., Dislodged load, Animal).
urban_or_rural_area	Indicates whether the collision occurred in an urban or rural setting.
did_police_officer_attend_scene_of_accident	Indicates if a police officer physically attended the scene (1 = Yes, 2 = No, 3 = Self-reported).

Accident(Casualty):

Column Name	Description
collision_index	A unique identifier linking the casualty to a specific collision.
vehicle_reference	Links the casualty to the specific vehicle they were in/on (or struck by, if a pedestrian).
casualty_reference	A unique sequential number for the casualty within a specific collision.
casualty_class	Defines the casualty's role (e.g., 1 = Driver/Rider, 2 = Passenger, 3 = Pedestrian).
sex_of_casualty	The gender of the casualty (1 = Male, 2 = Female, 9 = Unknown).
age_of_casualty	The age of the casualty in years.
casualty_severity	The severity of the casualty's injuries (1 = Fatal, 2 = Serious, 3 = Slight).
pedestrian_location	The location of the pedestrian at the time of impact (e.g., On a crossing, In carriageway).
pedestrian_movement	The direction and nature of the pedestrian's movement (e.g., Crossing from nearside).
car_passenger	Indicates if the casualty was a passenger in a car and their seating position.
casualty_type	The type of casualty, usually mirroring the vehicle_type they were associated with.
casualty_distance_banding	A banded distance representing how far the casualty lived from the collision site.

Vehicle:

Column Name	Description
collision_index	A unique identifier linking the vehicle to a specific collision.
vehicle_reference	A unique sequential number for the vehicle within a specific collision.
vehicle_type	The type of vehicle (e.g., Pedal cycle, Motorcycle, Car, Bus).
vehicle_manoeuvre	What the vehicle was doing immediately before the collision (e.g., Turning right, Reversing).
vehicle_direction_from	The compass direction the vehicle was traveling from.
vehicle_direction_to	The compass direction the vehicle was traveling to.
vehicle_location_restricted_lane	Indicates if the vehicle was in a restricted lane (e.g., Bus lane, Cycle lane).
junction_location	The position of the vehicle in relation to the junction at the time of impact.
skidding_and_overturning	Indicates if the vehicle skidded, jack-knifed, or overturned.
hit_object_in_carriageway	Indicates if the vehicle struck an object in the road (e.g., Bridge roof, Bollard).
vehicle_leaving_carriageway	Indicates if and how the vehicle left the roadway (e.g., Nearside, Offside).
hit_object_off_carriageway	Indicates if the vehicle struck an object off the road (e.g., Tree, Wall).
first_point_of_impact	The initial point of impact on the vehicle (e.g., Front, Back, Offside).
vehicle_left_hand_drive	Indicates whether the vehicle is a left-hand drive model.
sex_of_driver	The gender of the driver (1 = Male, 2 = Female, 3 = Not known).
age_of_driver	The age of the driver in years.
engine_capacity_cc	The engine capacity of the vehicle in cubic centimeters (CC).
propulsion_code	The type of fuel or propulsion used (e.g., Petrol, Heavy oil, Electric).

age_of_vehicle	The age of the vehicle in years from its first registration.
generic_make_model	The recorded make and model of the vehicle.
escooter_flag	A flag indicating whether the vehicle is an e-scooter.
driver_distance_banding	A banded distance representing how far the driver lived from the collision site.